

Chapter 1C:

Designing Intelligent Agents

AMCS2104 Fundamentals of Artificial Intelligence

Content

- Task Environment
- PEAS Specification
- Properties of Environments
- Agent Programs
- Simple Reflex Agents
- Model-based Reflex Agents
- Goal-based Agents
- Utility-based Agents
- Learning Agents
- Representation of Environment States

Task Environment

- A **task environment** represents the problems to which a rational agent is going to solve.
- Therefore, the first step in designing an agent, must always be to **specify the task environment as fully as possible**.
- The **PEAS** description can be used to describe a task environment in terms of:
 - "**P**"erformance measure
 - "**E**"nvironment
 - "**A**"ctuators
 - "**S**"ensors



PEAS Descriptions: Examples

Agent Type	Performance Measure	Environment	Actuators	Sensors
Taxi driver	Safe, fast, legal, comfortable trip, maximize profits, minimize impact on other road users	Roads, other traffic, police, pedestrians, customers, weather	Steering, accelerator, brake, signal, horn, display, speech	Cameras, radar, speedometer, GPS, engine sensors, accelerometer, microphones, touchscreen

PEAS Descriptions: Examples

Agent Type	Performance Measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments	Touchscreen/voice entry of symptoms and findings
Satellite image analysis system	Correct categorization of objects, terrain	Orbiting satellite, downlink, weather	Display of scene categorization	High-resolution digital camera

PEAS Descriptions: Examples

Agent Type	Performance Measure	Environment	Actuators	Sensors
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, tactile and joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, raw materials, operators	Valves, pumps, heaters, stirrers, displays	Temperature, pressure, flow, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, feedback, speech	Keyboard entry, voice

PEAS Descriptions: Examples

Agent Type	Performance Measure	Environment	Actuators	Sensors
Vacuum Cleaner	?	?	?	?

Properties of The Environment

- The range of environments that might arise in A.I. is obviously vast.
- Therefore, they are categorized into a number of dimensions to help **determine** the **appropriate agent design** and the **applicability** of each of the principal families of **techniques for agent implementation**:
 - Observability
 - Agents
 - Deterministic
 - Episodic
 - Static
 - Discrete

Fully Observable vs. Partially Observable

- An environment is **fully observable** if the agent's sensors give it access to the complete state of the environment at each point in time.
- Otherwise, the environment is **partially observable**.
- An environment might be partially observable due to **noisy** and **inaccurate sensors** or parts of the state are simply missing from the sensor data.
- Fully observable environments are convenient because agents need not keep track of the world.
- An agent without any sensor will always be situated in a **fully unobservable environment**.

Fully Observable vs. Partially Observable

Description	Fully Observable	Partially Observable
Access to the environment state at each point in time.	Full state	Partial state
Detection of any aspects relevant to the choice of action.	All	Some
Maintain any internal state to keep track of the world.	No	Some
Handling of any uncertainty that might arise.	No	Yes

Fully Observable vs. Partially Observable

□ Fully observable environment

- Chess game
- Image recognition

□ Partially observable environment

- Card game
- Taxi driving

Single-Agent vs. Multiagent

- In a single-agent environment, only one agent is involved and operating in it.
- In a multiagent environment, some kind of interactions happen between agents.
- **Communication** often emerges as a rational behavior in multiagent environments.
- Multiagent environment can be:
 - Competitive
 - Cooperative / Collaborative
 - Partially both

Single-Agent vs. Multiagent

Description	Single-Agent	Multiagent
The number of agent involved in an environment.	One	More than one
Distinguishing other agents from objects.	N/A	Yes

□ Single-agent environment:

- Crossword puzzle
- Spider card game

Multiagent Environment

□ **Competitive**

- Agents compete against each other.
- The gain of one agent is at the loss of others'.
- **Example:** Chess game

□ **Cooperative (or collaborative)**

- Agents cooperate to produce the desired output where everyone receives reward at each time step.
- **Example:** A rescue operation conducted by robots

Multiagent Environment

□ Partially both

- Multiple agents compete and cooperate with each other at the same time.
- This may happen due to the use of more than one performance measure.
- **Example:** Taxi driving and soccer

□ Issue

- Many entities might exist in an environment.
- **Question:** How to determine whether an entity is an agent or non-agent?
- The key distinction is whether B's behavior is best described as maximizing a performance measure whose value also depends on agent A's behavior.

Deterministic vs. Non-Deterministic

- An environment is **deterministic** if the next state of the environment is **completely determined** by the current state and the action executed by the agent(s).
- Otherwise, it is **non-deterministic**.
- In principle, an agent need not worry about **uncertainty** in a **fully observable** and **deterministic environment**.

Deterministic vs. Non-Deterministic

Description	Deterministic	Stochastic
The next environment state is completely determined by the current state and the action executed by the agent .	Yes	No
Any action has a single guaranteed effect.	Yes	No
Need to deal with uncertainty.	No	Yes

Deterministic vs. Non-Deterministic

□ Deterministic environment

- Chess game
- Tic-tac-toe game

□ Stochastic environment

- Soccer
- Taxi driving

Episodic vs. Sequential

- In an **episodic** task environment, the agent's experience is divided into atomic episodes.
- In each episode the agent receives a percept and then performs a single action, and the next episode does not depend on the actions taken in previous episodes.
- Therefore, agents working in an episodic environment do not need to think ahead.
- In **sequential** environments, on the other hand, the current decision could affect all future decisions.

Episodic vs. Sequential

Description	Episodic	Sequential
Agent's current decision affects all future decisions.	No	Yes
Keep the memory of past actions for future references.	No	Yes

Episodic vs. Sequential

□ Episodic environment

- Mail sorting system
- Defect detection assembly line

□ Sequential environment

- Chess game
- Taxi driving

Static vs. Dynamic

- A **static** environment is completely unchanged while an agent is perceiving the environment until the agent acts on it.
- A **dynamic** environment changes while an agent is deliberating.
- A **semi-dynamic** environment does not change with the passage of time but the agent's performance score does.

Description	Static	Dynamic
The environment doesn't change over time, only the agent's action modify it.	Yes	No

Static vs. Dynamic

□ **Static environment**

- Crossword puzzle
- Chess game, played without a clock

□ **Dynamic environment**

- Taxi driving
- Soccer

□ **Semi-Dynamic environment**

- Chess game, played with a clock

Discrete vs. Continuous

- The discrete / continuous distinction applies to:
 - The state of the environment.
 - The way time is handled.
 - The percepts and actions of the agent.
- A discrete environment has **a finite number of distinct states**, and there are **a finite number of percepts** and **actions** that can be performed within it.
- Otherwise, the environment is continuous.

Discrete vs. Continuous

Description	Discrete	Continuous
The environment has a [X] number of states.	Finite	Infinite
The agent has a [X] number of percepts.	Finite	Infinite
The agent has a [X] number of actions.	Finite	Infinite
There exist a [X] number of time points.	Finite	Infinite

Discrete vs. Continuous

□ Discrete environment

- Tic-tac-toe game
- Chess game

□ Continuous environment

- Basketball
- Taxi driving

Known vs. Unknown

- The distinction does not refer to the environment itself but the **agent's state of knowledge** about the "**laws of physics**" of the environment.
- In a **known** environment, the outcomes for all actions are given.
- However, in an **unknown** environment, an agent will have to learn how it works in order to make good decisions.
- A performance measure itself might be unknown, either because the designer is not sure how to write it down correctly or because the ultimate user—whose preferences matter—is not known.

Difficulty of Problems

- **Question:** Which type of environment is more difficult to deal with?
 - Fully observable vs. Partially observable
 - Single-agent vs. Multiagent
 - Deterministic vs. Non-deterministic
 - Episodic vs. Sequential
 - Static vs. Dynamic
 - Discrete vs. Continuous
 - Known vs. Unknown

Task Environments: Examples

Task Environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

The Structure of The Agent

- Suppose an agent program will run on **some sort of computing device with physical sensors and actuators** (or **architecture**), then the structure of an agent:

$$\text{Agent} = \text{Architecture} + \text{Program}$$

- In general, the **architecture**:
 - Makes the percepts from the sensors available to the program.
 - Runs the program (by the hardware boards and chips).
 - Feeds the program's action choices to the actuators as they are generated.
- The type of architecture needed depends on the type of problem to be solved.

Agent Program

□ Agent function

- An **abstract mathematical description** that maps any given **percept sequence** to an action.

□ Agent program

- A concrete **implementation** of the **agent function**, running within some physical system.
- **Note:** All **agent programs** provided in this chapter will be presented in a **simple pseudocode language**.

Table-Driven Agents

function TABLE-DRIVEN-AGENT(*percept*) **returns** an action
 persistent: *percepts*, a sequence, initially empty
 table, a table of actions, indexed by percept sequences, initially fully specified

 append *percept* to the end of *percepts*
 action $\leftarrow$ LOOKUP(*percepts*, *table*)
 return *action*

- This agent program represents a sample implementation of the agent function for the vacuum world problem discussed earlier.
- **Question:** This agent program does what we want it to do, is it a good one?

The Lookup Table: Agent Function

- A table that contains the appropriate action for every possible percept sequence.

Percept sequence	Action
<i>[A, Clean]</i>	<i>Right</i>
<i>[A, Dirty]</i>	<i>Suck</i>
<i>[B, Clean]</i>	<i>Left</i>
<i>[B, Dirty]</i>	<i>Suck</i>
<i>[A, Clean], [A, Clean]</i>	<i>Right</i>
<i>[A, Clean], [A, Dirty]</i>	<i>Suck</i>
<i>⋮</i>	<i>⋮</i>
<i>[A, Clean], [A, Clean], [A, Clean]</i>	<i>Right</i>
<i>[A, Clean], [A, Clean], [A, Dirty]</i>	<i>Suck</i>
<i>⋮</i>	<i>⋮</i>

Table-Driven Agents: Limitation

- Let **P** be the set of possible percepts and **T** be the lifetime of the agent, the lookup table will contain the following number of entries:

$$\sum_{t=1}^T |P|^t$$

- **Issues arise**

- How to store such a gigantic table?
- How much time is needed to create the table?
- How is it possible for an agent to learn from its experience?

- **Challenge:** How to write smallish programs that produce a rational behavior?

Different Types of Agent Program

- There are 4 basic kinds of **agent programs** that embody the principles underlying almost all intelligent systems:
 - Simple reflex agents
 - Model-based reflex agents
 - Goal-based agents
 - Utility-based agents
- Each of these agent program combines a number of different components in different ways to generate actions.

Simple Reflex Agents

- A **simple reflex agent** selects actions on the basis of the **current percept**, **ignoring the rest of the percept history**.
- A sample implementation of the vacuum agent as a simple reflex agent is provided below:

```
function REFLEX-VACUUM-AGENT([location,status]) returns an action  
    if status = Dirty then return Suck  
    else if location = A then return Right  
    else if location = B then return Left
```

Simple Reflex Agents

- A **reflex-vacuum-agent** program uses the **condition–action rule** (also called **situation–action rules**, or **if–then rules**) which can be written as an **if-else statement**.
- Example

IF [Dirty] THEN [SUCK]

- A **reflex-vacuum-agent** program is very small because:
 - It ignores the rest of the percept history.
 - It cuts down the number of possibilities from 4^T to 4.

Simple Reflex Agents

- A generic version of the this agent program.

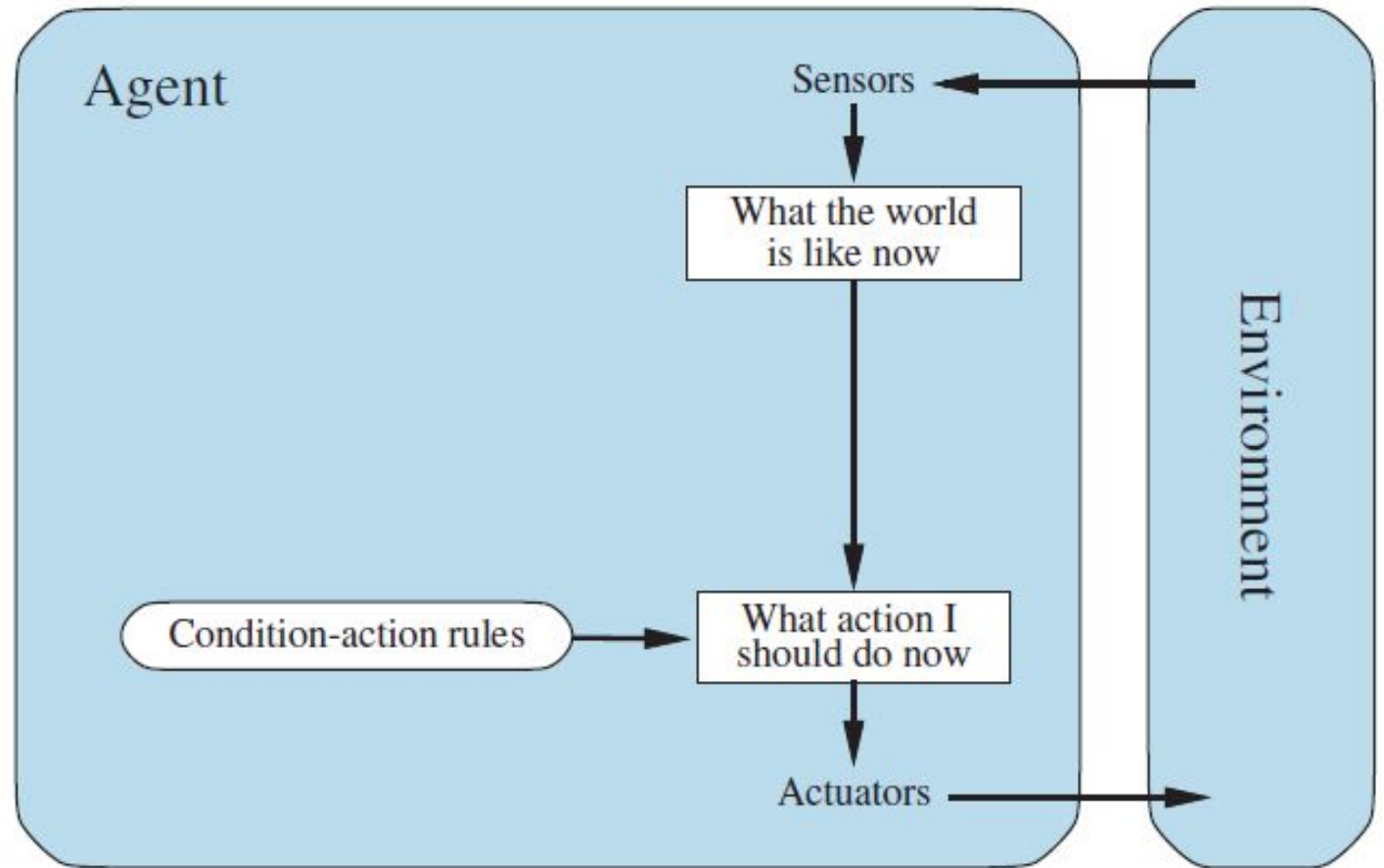
```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition–action rules

  state ← INTERPRET-INPUT(percept)
  rule ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

INTERPRET-INPUT generates an abstracted description of the current state from the percept.
RULE-MATCH returns the first rule in the set of rules that matches the given state description.

Simple Reflex Agents

- A **simple reflex agent** makes a connection from percept to action based on the **condition–action rules**.



Simple Reflex Agents: Limitation

□ Pros

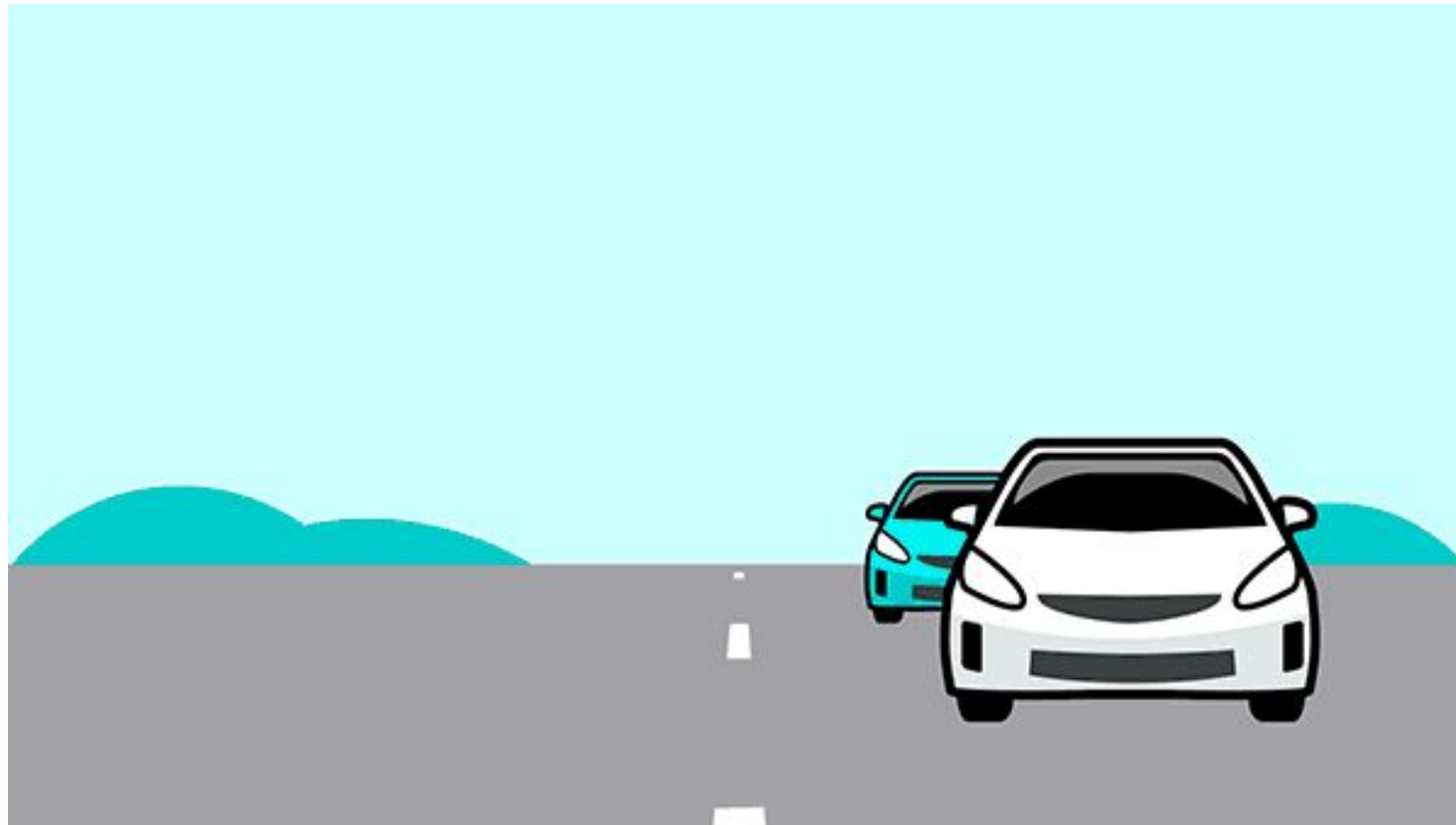
- Easy to implement.

□ Cons

- Only works well if the environment is **fully observable**.
 - A simple reflex vacuum agent that is deprived of its location sensor and has only a dirt sensor will perform badly.
 - Vulnerable to getting stuck in an infinite loop.
-
- However, a **randomized** simple reflex agent outperforms a simple reflex agent.

Model-based Reflex Agents

□ **Question:** How can an agent handle its partially observability?



Model-based Reflex Agents

- One solution is to keep track of **the part of the world it can't see now**.
- **Model-based reflex agent**
 - Maintains internal states (that depends on the percept history) to reflect some unobserved aspects of the current state.
 - Updates this internal state from time to time.
- 2 types of knowledge have to be encoded in the agent program:
 - The sensor model
 - The transition model

Model-based Reflex Agents

□ Sensor Model

- A knowledge that reflects the state of the world.

□ Transition Model

- A knowledge of how the world changes over time:
 - How the world evolves independently of the agent?
 - How the world evolves under the effects of the agent's actions?

- A model-based agent uses such models to **keep track of the state of the world.**

Model-based Reflex Agents

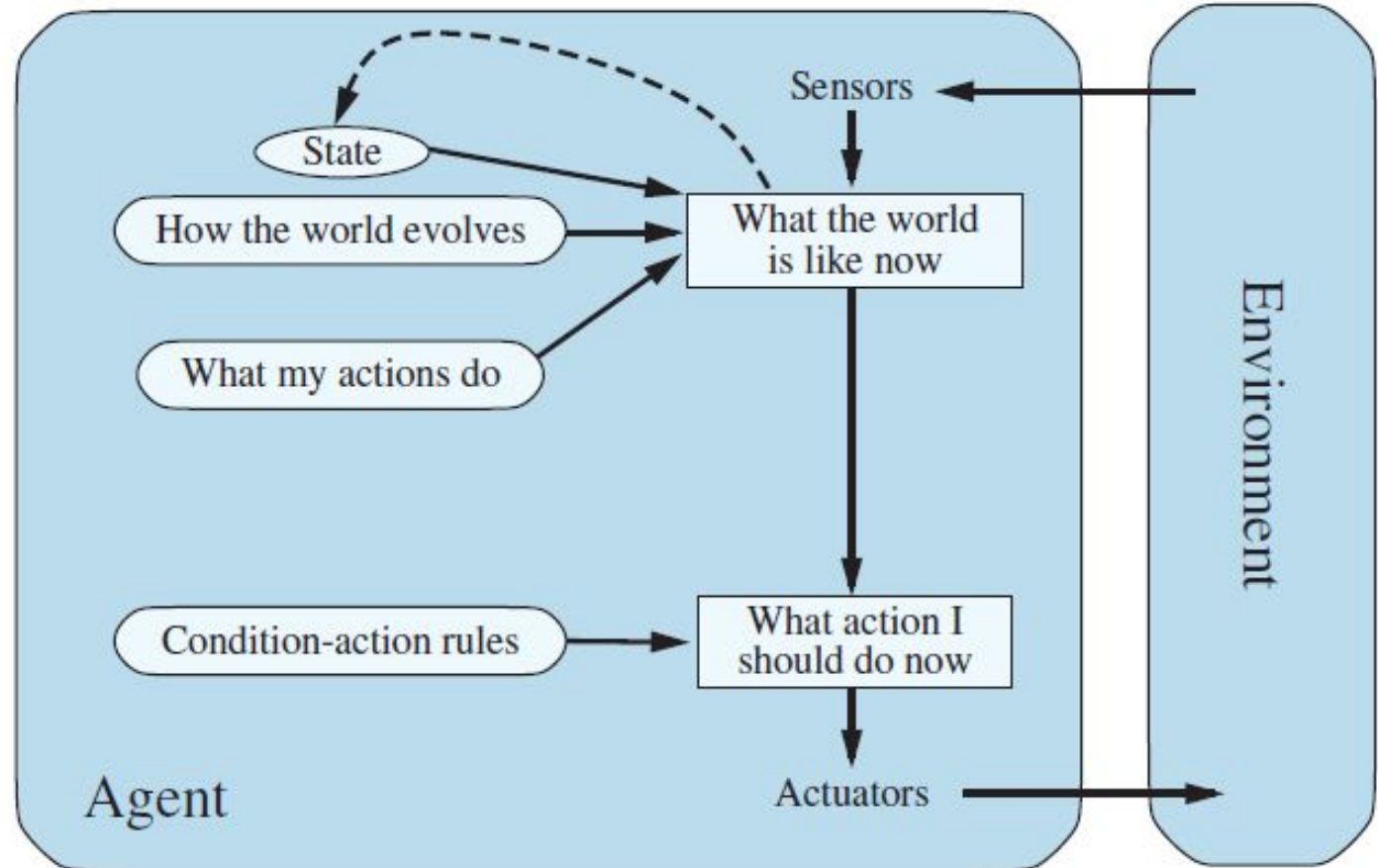
function MODEL-BASED-REFLEX-AGENT(*percept*) **returns** an action
 persistent: *state*, the agent's current conception of the world state
 transition_model, a description of how the next state depends on
 the current state and action
 sensor_model, a description of how the current world state is reflected
 in the agent's percepts
 rules, a set of condition–action rules
 action, the most recent action, initially none

state $\leftarrow$ UPDATE-STATE(*state*, *action*, *percept*, *transition_model*, *sensor_model*)
rule $\leftarrow$ RULE-MATCH(*state*, *rules*)
action $\leftarrow$ *rule*.ACTION
return *action*

UPDATE-STATE creates a new internal state description that tells the agent "What the world is like now".

Model-based Reflex Agents

- A **model-based reflex agent** generates the updated description of the current state, based on the **agent's model of how the world works**.



Model-based Reflex Agents: Limitation

□ Cons

- The internal state can only represent the agent's best guesses.
 - The agent still fails to determine the current state of a partially observable environment exactly.
 - Yet, the agent still have to **make a decision under uncertainty**.
-
- Without having a goal that describes situations that are desirable, the agent may not know what to do when more than one action can be carried out in certain environment states.

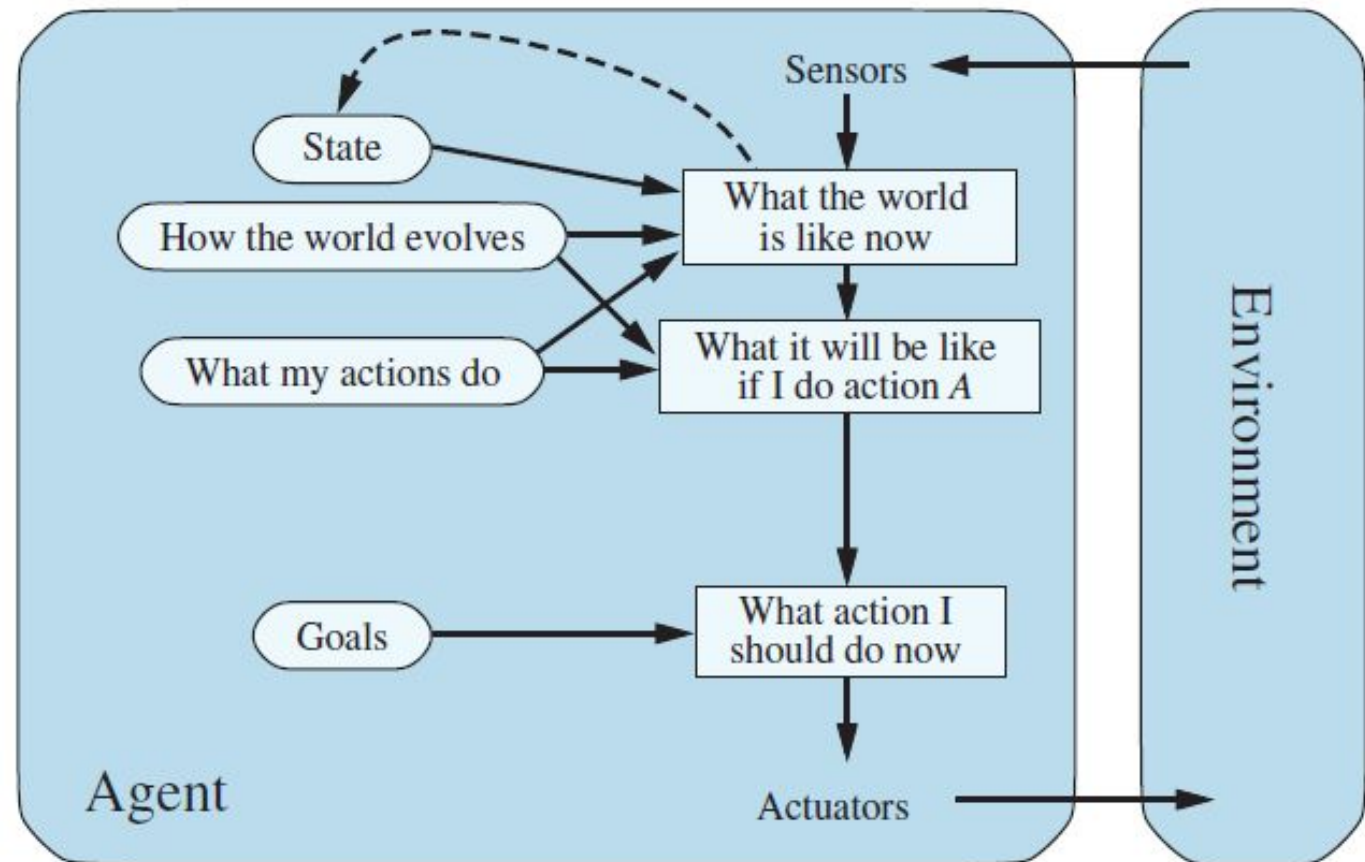
Goal-based Agents

- A **goal-based agent** keeps track of some **goal information** that describes situations that are desirable, which then helps the agent in choosing the right action.
- **Goal-based agents vs. reflex agents**

Description	Goal-based Agents	Reflex Agents
Decision making	Reasoning	Reflection
Adaptability	Higher	Lower

Goal-based Agents

- A **goal-based agent** keeps track of the **world state** and a **set of goals** it is trying to achieve, and chooses an action that will (eventually) lead to the achievement of its goals.



Goal-based Agents: Limitation

□ Cons

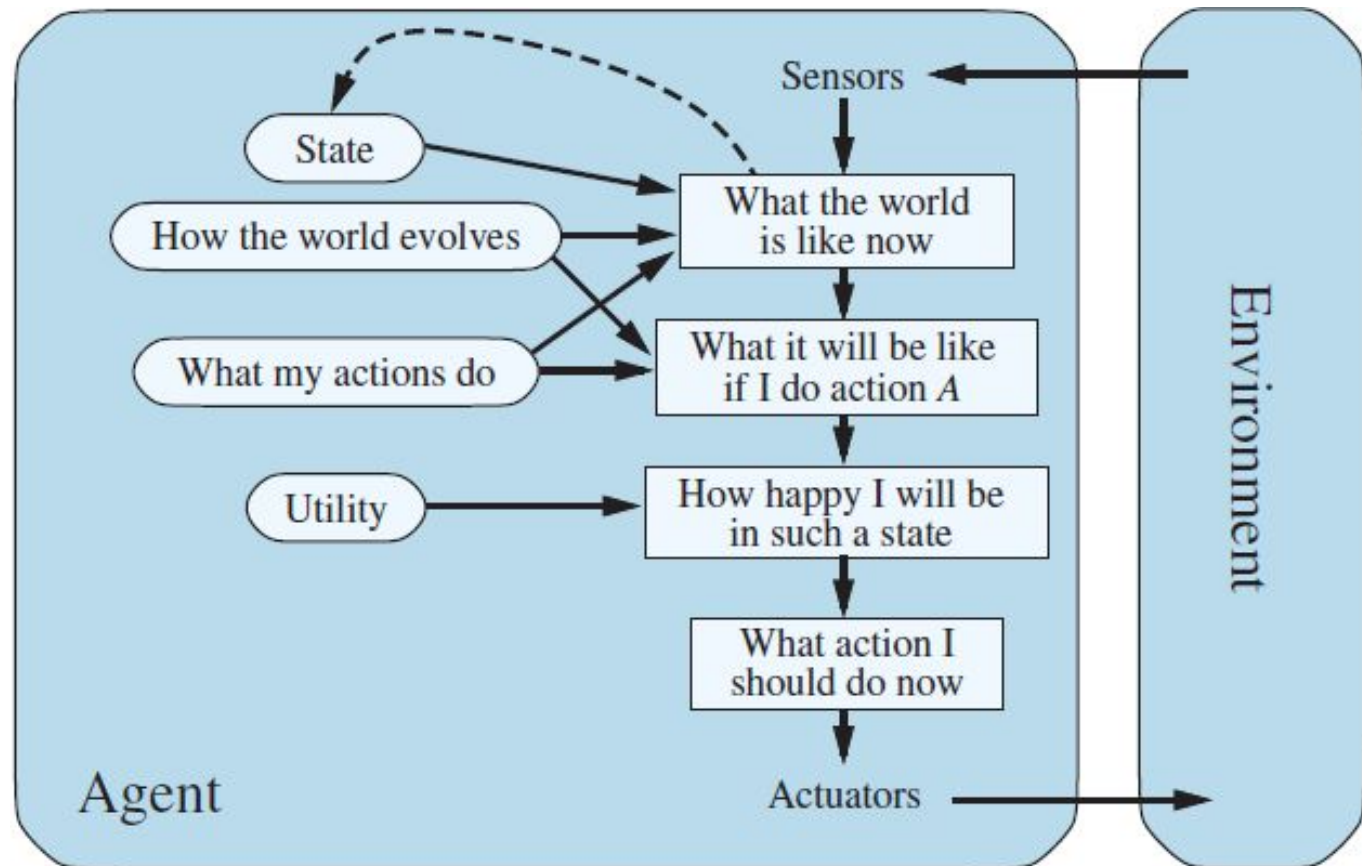
- Goal information can only provide a **binary distinction** between whether it is achievable or not.
- Despite uncertainty about which sequence of world states are more desirable, the agent still has to make a decision.

Utility-based Agents

- A **utility function** assigns score to different world states according to exactly "how happy they would make the agent".
- A utility function is different from the performance measure:
 - **Utility function**: Internal performance measure.
 - **Performance measure**: External performance measure.
- If the internal utility function and the external performance measure are in agreement, then a **rational agent** will choose actions that maximize its utility.

Utility-based Agents

- A **utility-based agent** chooses the action that leads to the **best expected utility** by averaging over all possible outcome states weighted by the probability of the outcome.



Utility-based Agents

Can a rational decision be made when...?	Utility-based Agents	Goal-based Agents
There is more than one goal to be achieved.	Yes	No
There are conflicting goals.	Yes	No
The environment is uncertain (partially observable).	Yes	No

Utility-based Agents: Challenges

- **Building intelligent agents** that maximize expected utility is very challenging as:
 - Agents need to model and keep track of complicated environments.
 - Developing such models involves a great deal of research on perception, representation, reasoning and learning.
 - Developing utility-function requires the use of effective and efficient algorithms.
 - Solutions with high computational complexity are not practical.

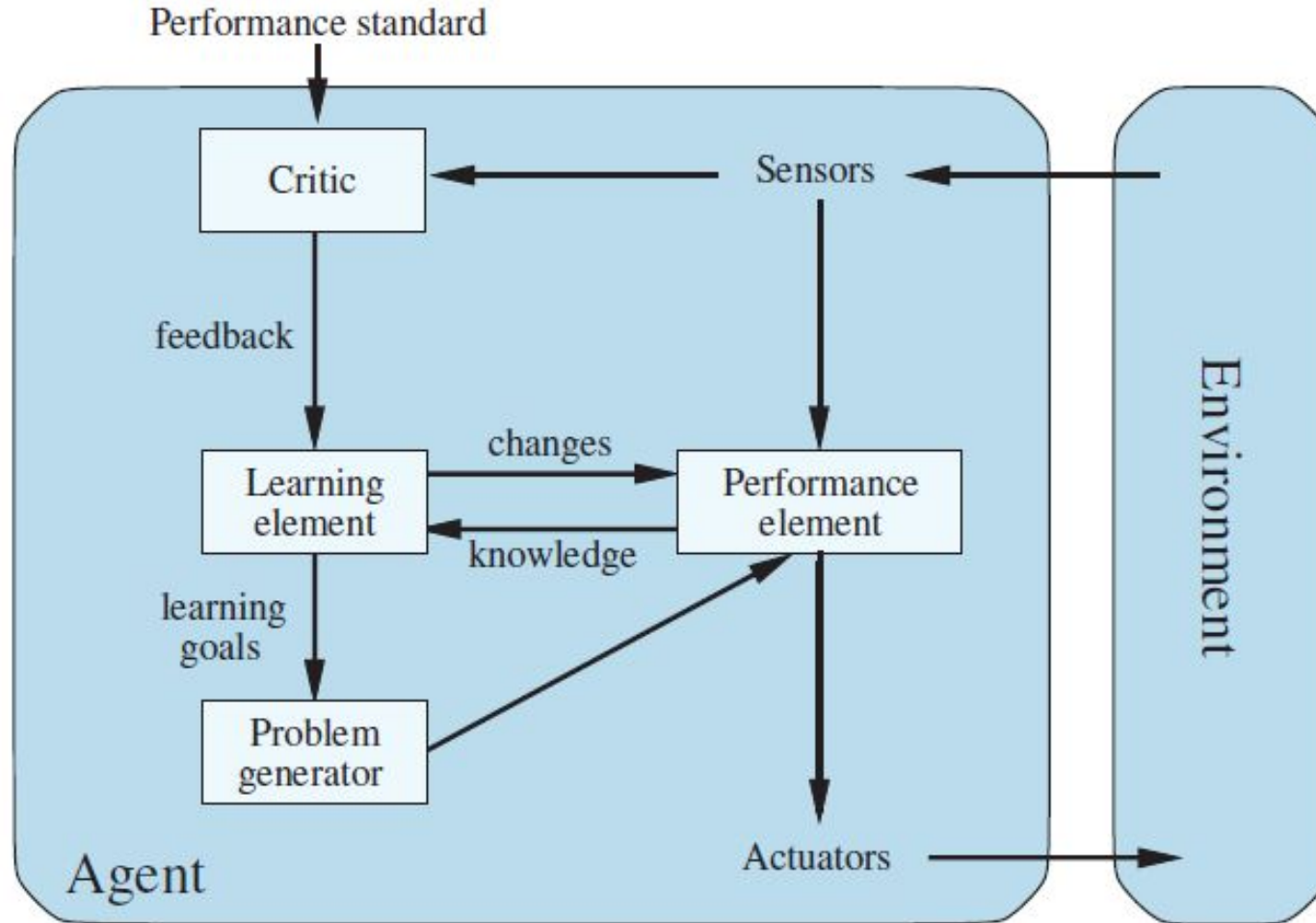
Learning Capability

- In 1950, Alan Turing proposed a method of programming intelligent machines, that is, build **learning machines** and then to teach them.
- Any type of agent can be built as a learning agent.
- Being able to learn is a **competitive advantage** because it allows the agent to operate in initially unknown environments and to become more competent than its initial knowledge alone might allow.

Learning Agents

- A **learning agent** can be divided into 4 conceptual components:
 - **Performance element (agent)**
 - Responsible for selecting external actions.
 - **Learning element**
 - Responsible for making improvements.
 - **Critic**
 - Responsible for evaluating the agent's behavior with respect to a fixed performance standard.
 - **Problem generator**
 - Responsible for suggesting actions that will lead to new experiences.

Learning Agents



Automated Taxi: Example

□ Performance element

- A collection of knowledge and procedures the taxi has for selecting its driving actions on the road.

□ Critic

- Observes the world (changes) and provides feedback to learning element in accordance with a fixed performance standard.

□ Learning element

- Amend the performance element in accordance with the feedback.

□ Problem generator

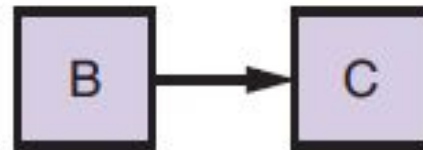
- Suggest "experiments" on certain areas of behavior for further improvement.

Representation

- An agent program consists of various components that model the environment the agent inhabits.
- Example
 - **Sensor Model:** What the world is like now?
 - **Transition Model:** What my actions do?
- These components can be represented in various ways (in increasing complexity and expressive power):
 - Atomic representation
 - Factored representation
 - Structured representation

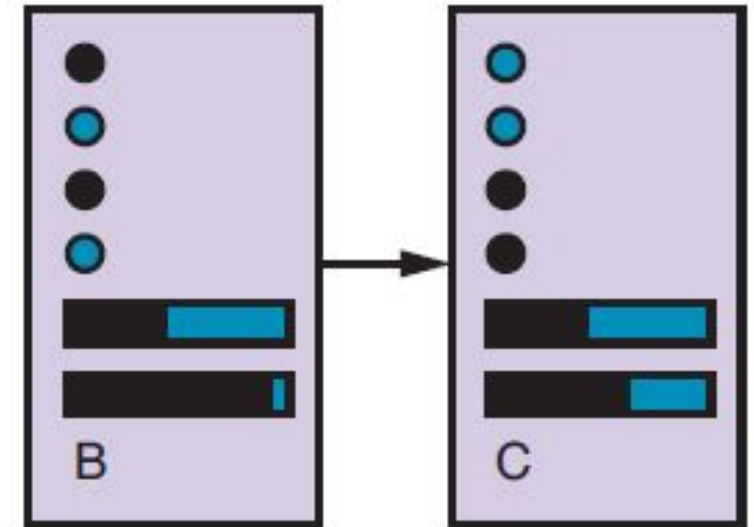
Atomic Representation

- In an **atomic representation**, each state of the world is indivisible without any internal structure.
- Example: State B and State C
 - Consider the problem of finding a driving route from one city to another via some sequence of cities.
 - Two cities can be represented by two atomic state B and state C.



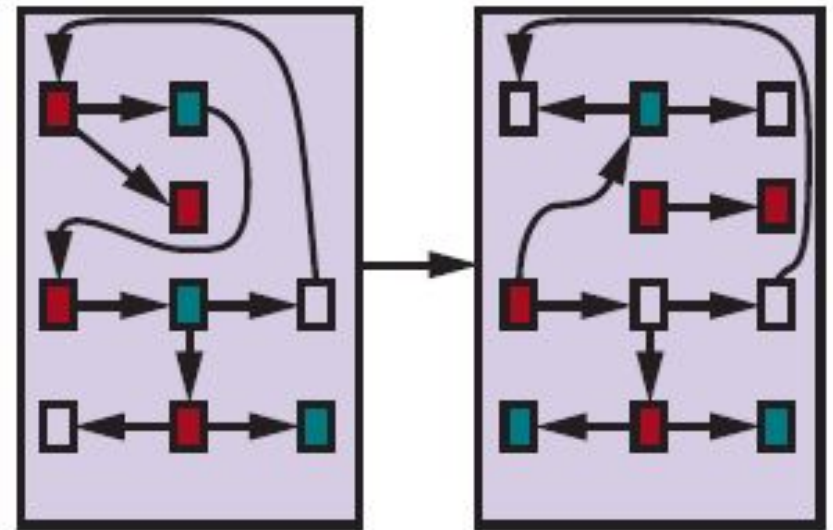
Factored Representation

- A **factored representation** splits up each state into a fixed set of **variables** or **attributes**, each of which **can have a value**.
- Example: State B and State C
 - Two different factored states can have:
 - Common attributes: Same GPS location.
 - Different attributes: Toll charges.



Structured Representation

- The world has many things in it that are related to each other.
- For many purposes, we need to understand the relationship between them, not just variables with values.
- In a **structured representation**, a state includes objects, each of which may have attributes of its own as well as relationships to other objects.



Structured Representation

□ Example

- Suppose we notice that a large truck ahead of us is reversing into the driveway of a dairy farm, but a loose cow is blocking the truck's path...
- A **factored representation** is unlikely to be pre-equipped with the attribute **TruckAheadBackingIntoDairyFarmDrivewayBlockedByLooseCow** with value true or false.
- With a **structured representation**, objects such as cows and trucks and their various and varying relationships can be described explicitly.

Reading Homework

- **Artificial Intelligence: A Modern Approach**, 4th Edition, by Stuart Russell and Peter Norvig
 - Chapter 2: Intelligent Agents (page 61 to page 77)

The End